

AI Storage Infra, Reliability

CS6501: AI Systems

Fall 2026

Lecture 7

Yue Cheng



UNIVERSITY
of VIRGINIA

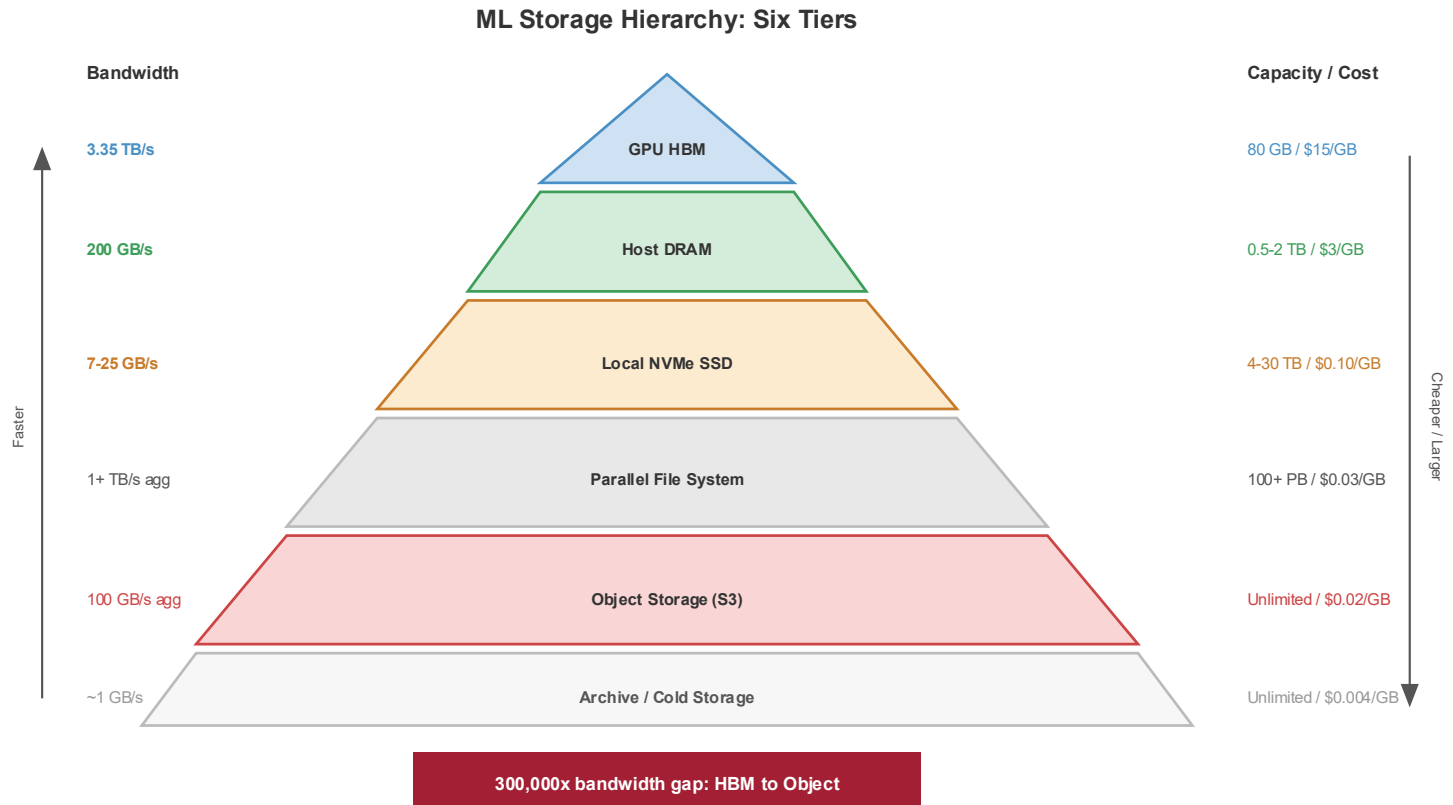
Material taken/derived from:

- Harvard ML Systems by Vijay Janapa Reddi

@ 2026 released for use under a [CC BY-SA](#) license.

Storage hierarchy

The six-tier AI storage hierarchy



Each tier trades bandwidth for capacity and cost. Physics of distance governs the tradeoff.

Design principle: Right data, right tier, right time. Each tier is a prefetch buffer for the tier above.

Bandwidth cliffs between tiers

Tier	Storage	Bandwidth	Cliff
0	GPU HBM	3.35 TB/s	↓ 17×
1	Host DRAM	200 GB/s	↓ 15×
2	Local NVMe	7–25 GB/s	↓ 3–10×
3	Parallel FS	4 GB/s/node	↓ 10×
4	Object Storage	~1 GB/s/node	

Implication: HBM empties in 24 ms but takes 400 ms to refill from DRAM. The next batch must *already* be in HBM before the current one finishes. Every tier prefetches for the tier above.

Tier 0: GPU HBM – The destination

HBM is the primary tier for GPU computation, holding active weights and activations.

Component (per GPU)	Size
Weights (FP16, tensor-parallel)	43.75 GB
Optimizer (ZeRO-3)	5.5 GB
Activations	10–20 GB
Gradient buffers	5.5 GB
NCCL buffers	2–4 GB
Total occupied	67–79 GB

H100: 80GB HBM

85—98% occupied

<15% free for prefetch

Late batch = GPU stall

Early batch = no room

Just-in-time, not just-too-late

Predict: Text vs. image bandwidth

Q: A 2,048-GPU cluster with 200 ms iteration time

Estimate the aggregate storage bandwidth needed for:

- a) Text** training (4,096 tokens/GPU, 4 bytes/token)
- b) Image** training (256 images/GPU, 150 KB/image)

Reveal: Text vs. image bandwidth

	Text	Image	Ratio
Batch size/GPU	4,096 tokens $\times$ 4 B	256 imgs $\times$ 150 KB	
Per-GPU data	16 KB	38.4 MB	2,400 $\times$
Aggregate BW	160 MB/s	393 GB/s	2,400 $\times$
Storage needed	Single NAS node	High-perf PFS	

Text: bandwidth-light, volume-heavy. Bottleneck = dataset size (large corpora) & checkpoint

Image: bandwidth-heavy. Bottleneck = storage system's ability to feed accelerators

Pipeline equation

The data pipeline equation

$$BW_{\text{req}} = N_{GPUs} \times U_{\text{target}} \times \frac{S_{\text{batch}}}{T_{\text{iteration}}}$$

Pipeline equation: Four levers

Variable	Meaning
N_{GPUs}	Number of accelerators
U_{target}	Target utilization (0.8–0.95)
S_{batch}	Local batch size (bytes)
$T_{iteration}$	Forward-backward time

Key: No lever is free. Reducing any input has a cost.

Worked example (ImageNet, 256 GPUs):

$$\begin{aligned} & 256 \times 0.80 \times \frac{256 \times 150KB}{0.2s} \\ &= 256 \times 0.80 \times 192 \text{ MB/s} \\ &= \mathbf{39.3 \text{ GB/s}} \end{aligned}$$

Must sustain **39.3 GB/s** continuously for days.
Fall short → accelerators idle.

The data stall ratio

$$\text{Data Stall \%} = \frac{T_{step} - T_{compute}}{T_{step}} \times 100$$

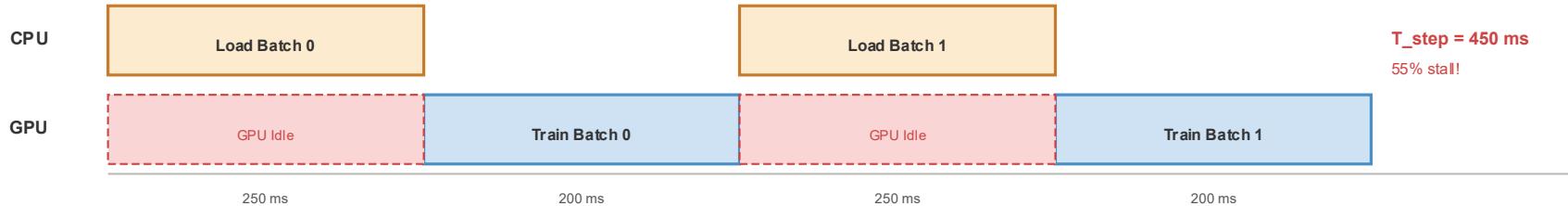
Stall Ratio	Diagnosis	Action
< 2%	Healthy pipeline	Monitor, do not over-invest
2–10%	Marginal (will break at scale)	Audit pipeline, add prefetch
> 10%	Clear storage bottleneck	Redesign data path

Economic consequence: 5% stall on 1,000 GPUs at \$2/GPU-hr for 1 week = **\$16,800 wasted**. Storage monitoring is a first-order economic concern.

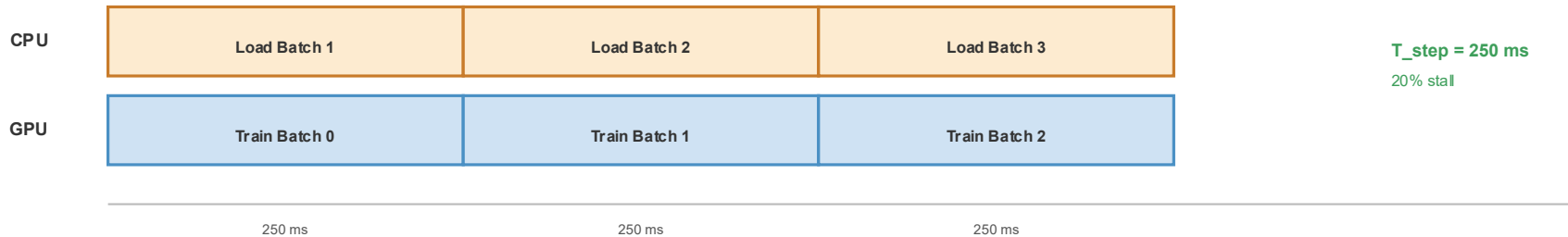
Pipelining and prefetching

Pipelined Data Loading: Hiding Storage Latency

Without Pipelining (Sequential)



With Pipelining (Overlapped)



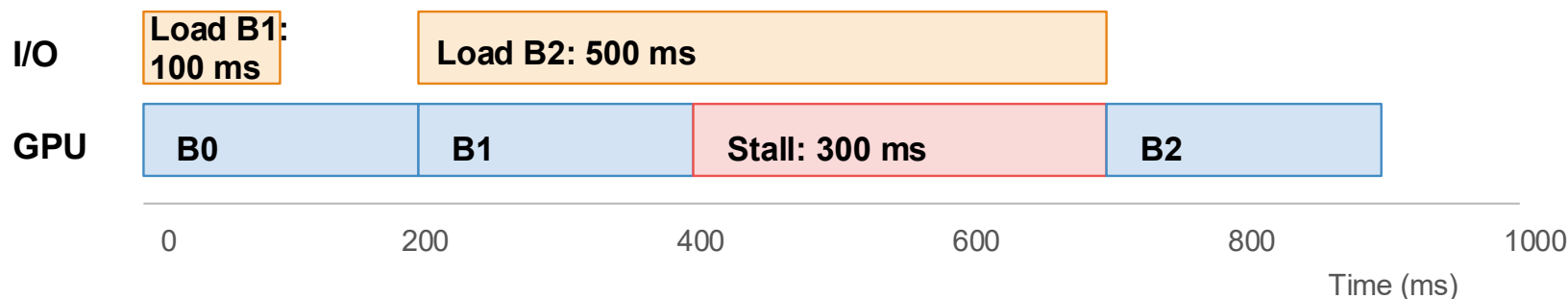
Pipelining: 450 ms to 250 ms (1.8x faster)

Prefetching buffers absorb tail latency

Same 700 ms load delay; 200 ms compute per batch

1 ready batch (B1)

300 ms GPU stall

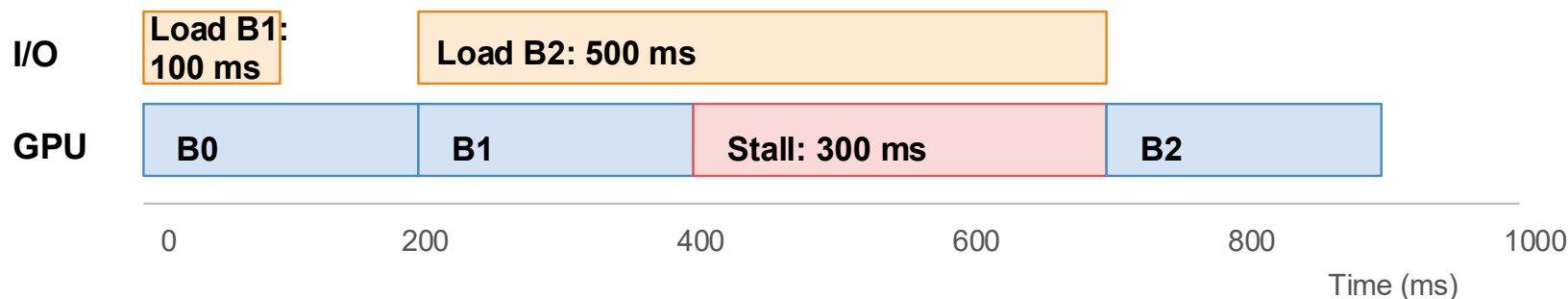


Prefetching buffers absorb tail latency

Same 700 ms load delay; 200 ms compute per batch

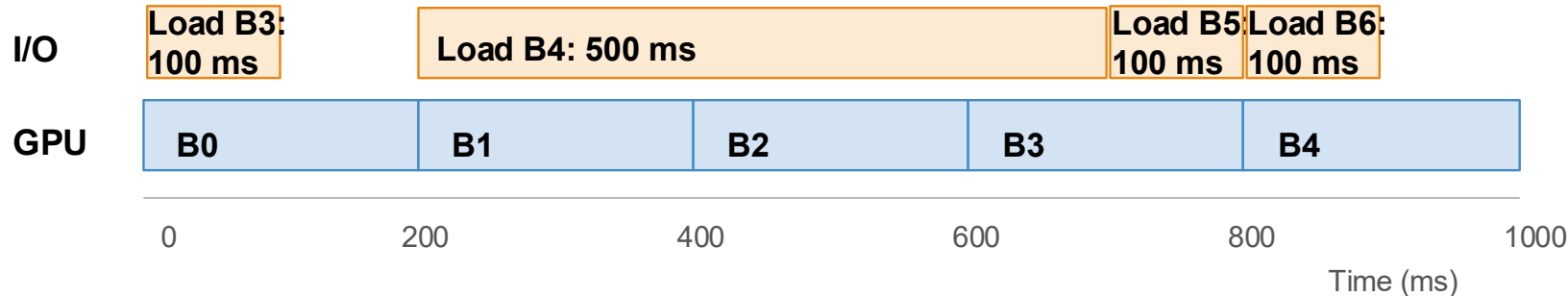
1 ready batch (B1)

300 ms GPU stall



3 ready batches (B1–B3)

No GPU stall



Ready batches keep the GPU busy during a slow load.

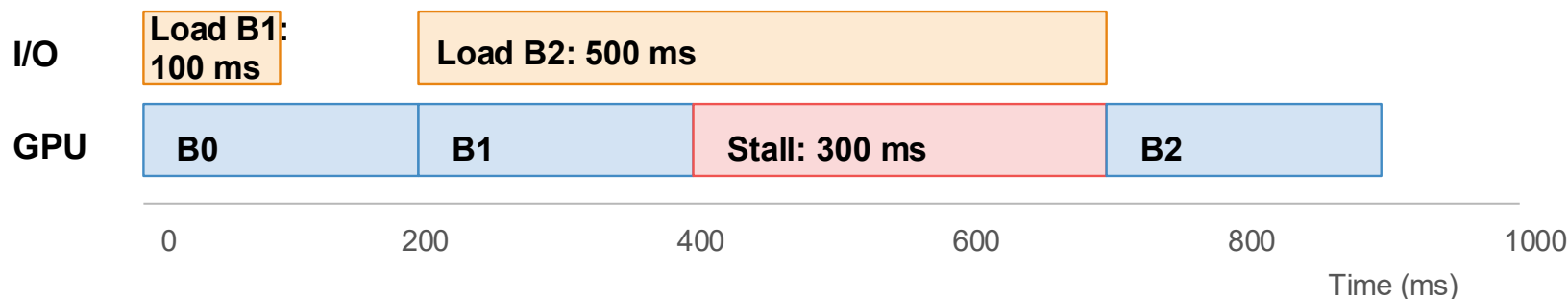
Buffer depth excludes the current batch and the in-flight load.

Prefetching buffers absorb tail latency

Same 700 ms load delay; 200 ms compute per batch

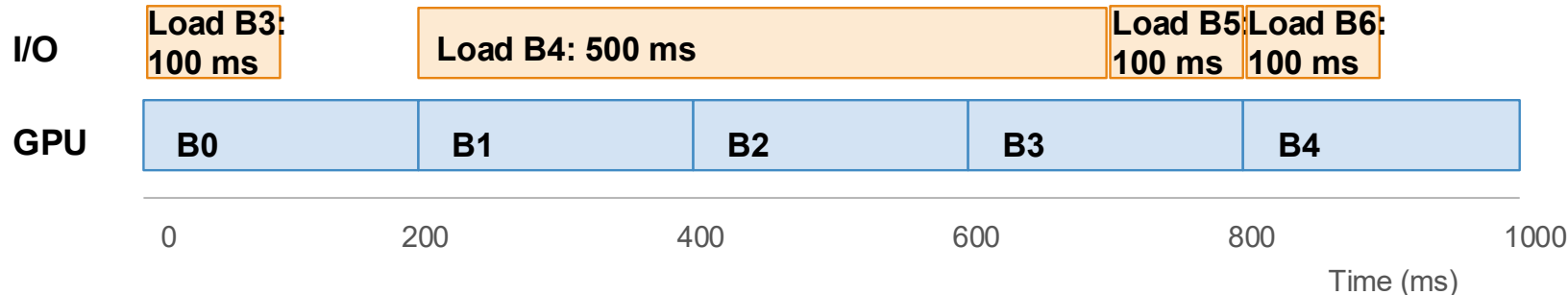
1 ready batch (B1)

300 ms GPU stall



3 ready batches (B1–B3)

No GPU stall



Critical caveat: Pipelining hides average latency. Buffer depth must absorb **P99 tail latency** to prevent stalls **at scale**.

Quick check

What is the 500× storage-compute gap, and what technique hides it?

Size the prefetch buffer

Size the prefetch buffer

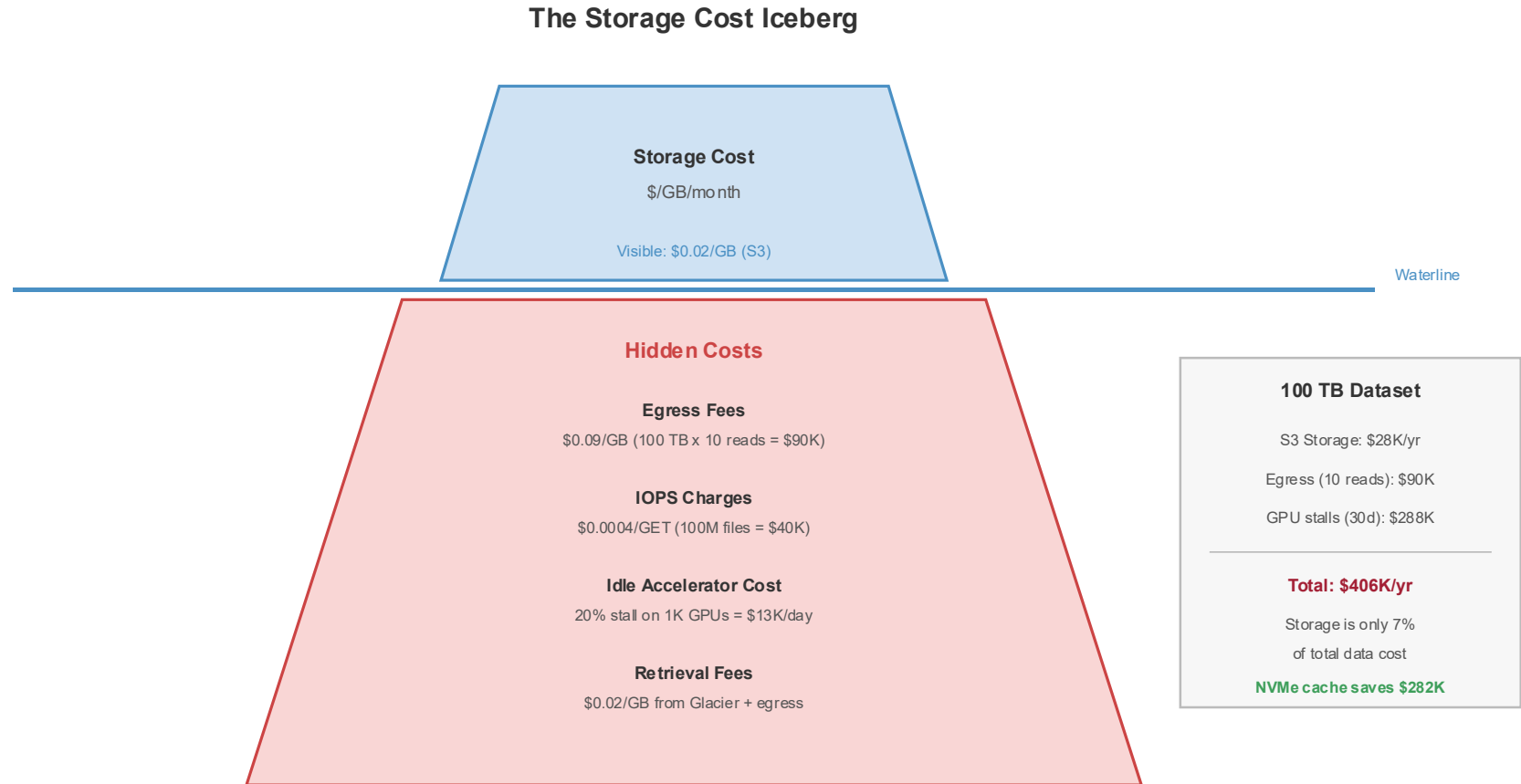
A training node has:

- $T_{\text{compute}} = 200$ ms per batch
- $T_{\text{I/O,avg}} = 250$ ms (avg load time)
- $T_{\text{I/O, P99}} = 500$ ms (tail load latency)

Calculate min prefetch depth (in batches) for
(a) average and (b) P99 latency.

Economics

The storage cost iceberg



Counterintuitive: The cheapest storage tier is not the cheapest choice. Total cost of data delivery includes egress + idle GPUs.

Worked example: Stream vs. Cache

50 TB vision dataset, 20 epochs, 4 training runs/year

Option A: Stream from S3

Storage: \$13,800/yr

Egress: $50 \text{ TB} \times 20 \times \$0.09/\text{GB} = \$90\text{K}/\text{run}$

4 runs/yr: \$360K egress

Total: \$373,800/yr

Option B: NVMe cache

S3 storage: \$13,800/yr

NVMe capacity: \$60,000/yr

Egress (stage 1x/run): \$18,000/yr

Total: \$91,800/yr

NVMe caching saves \$282,000/year: enough for 3 additional H100 GPUs.

Break-even: 4 epochs of reading per training run. Multi-epoch training always benefits from local caching.

Tiering strategy

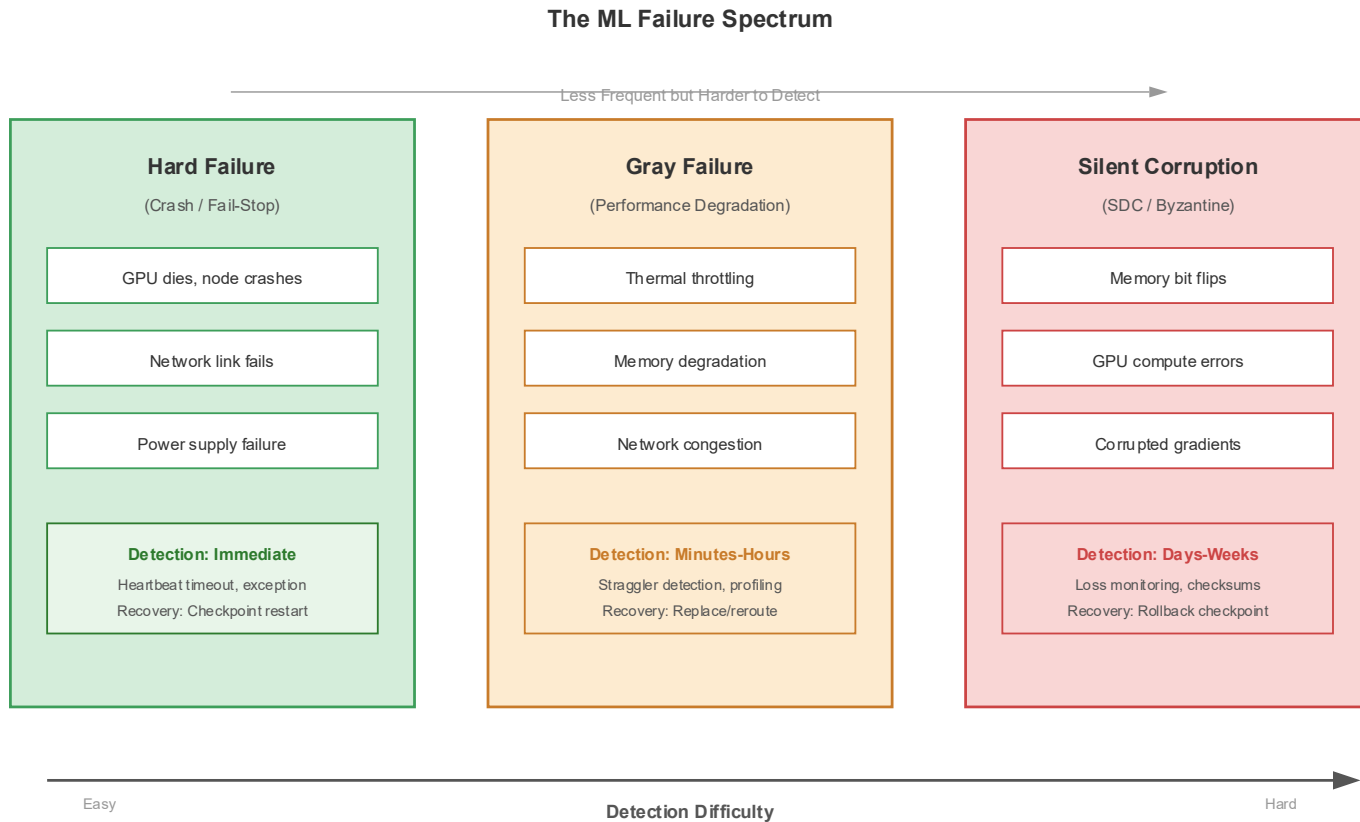
Tier	Storage	What Lives Here	\$/GB/mo
Hot	Local NVMe + DRAM	Active training data, current batch	\$0.10
Warm	Parallel FS	Shared datasets, recent checkpoints	\$0.03
Cold	Object (S3)	All organizational data, versioned	\$0.02
Archive	Glacier	Compliance, reproducibility	\$0.004

Lifecycle: Data promotes upward when a job requests it, demotes downward when idle. Automate with policies, not engineers.

Staging patterns: *Pre-stage* (best steady-state, slow start), *On-demand* (slow first epoch), *Streaming* (zero startup, no cache). Choice depends on epoch count and local NVMe capacity.

AI failures and checkpoints

AI failure spectrum



Fault tolerance must address all three categories: each requires different detection and recovery mechanisms.

Key: Hard failures are easy. Silent data corruption is the real threat at scale.

Hardware fault taxonomy

Fault Type	Behavior	Rate	Detection
Transient	Vanishes on retry (bit flip)	1 per 10K GPU-h	Silent
Permanent	Component dead (GPU, NIC, disk)	1 per 50K GPU-h	ECC/heartbeat
Intermittent	Works 99% of the time	1 per 25K GPU-h	Very hard
Byzantine	Sends incorrect data while “healthy”	1 per 100K GPU-h	Requires voting

At 10K GPUs: ~24 transient faults/day, 5 permanent/day. Every category occurs daily at scale.

Transient fault mechanism

Space

Galactic cosmic rays

Solar cosmic rays

Atmosphere

① Reactions in the atmosphere

High-energy particle

High-energy particle breaks nucleus

Nucleus(Nitrogen, Oxygen)

proton

neutron

π -meson

Muon

② Reactions in telecommunication equipment

Soft error

Temporary error (Recover by restart/overwrite etc.)

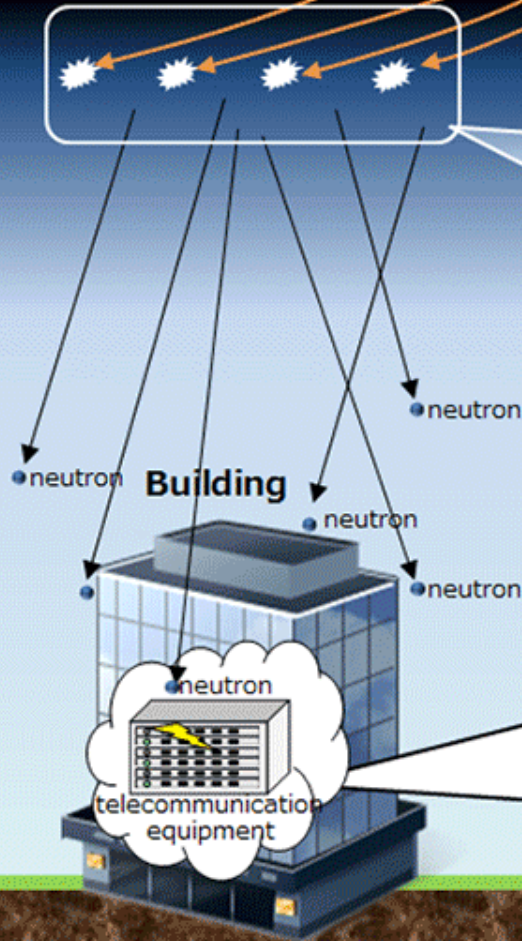
Electrical noises

Semiconductor device
in telecommunication
equipment

Neutron breaks
silicon nucleus

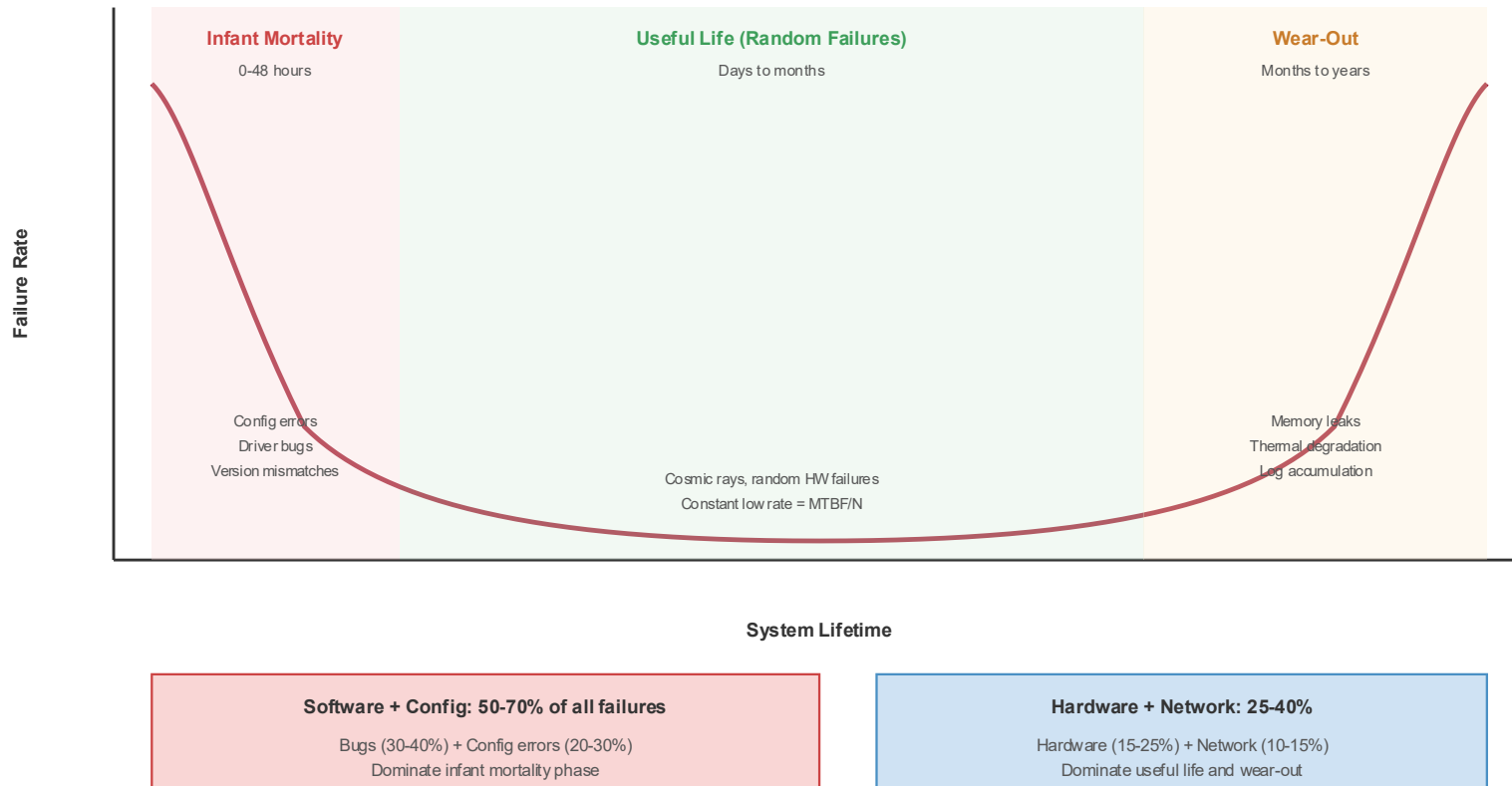
Silicon nucleus

Creating
secondary particles



Faults and the bathtub curve

The Bathtub Curve: Failure Rate Over System Lifetime



Most ML training runs experience infant mortality failures -- they rarely run long enough for wear-out

Economic case: \$10M/month cluster $\times$ 30% downtime = \$3M/month recoverable via fault tolerance.

The mathematics of inevitable failure

Individual component reliability:

$$R_{single}(t) = e^{-\lambda t}$$

For N identical components (series):

$$R_{system}(t) = e^{-N\lambda t}$$

The mathematics of inevitable failure

Individual component reliability:

$$R_{single}(t) = e^{-\lambda t}$$

For N identical components (series):

$$R_{system}(t) = e^{-N\lambda t}$$

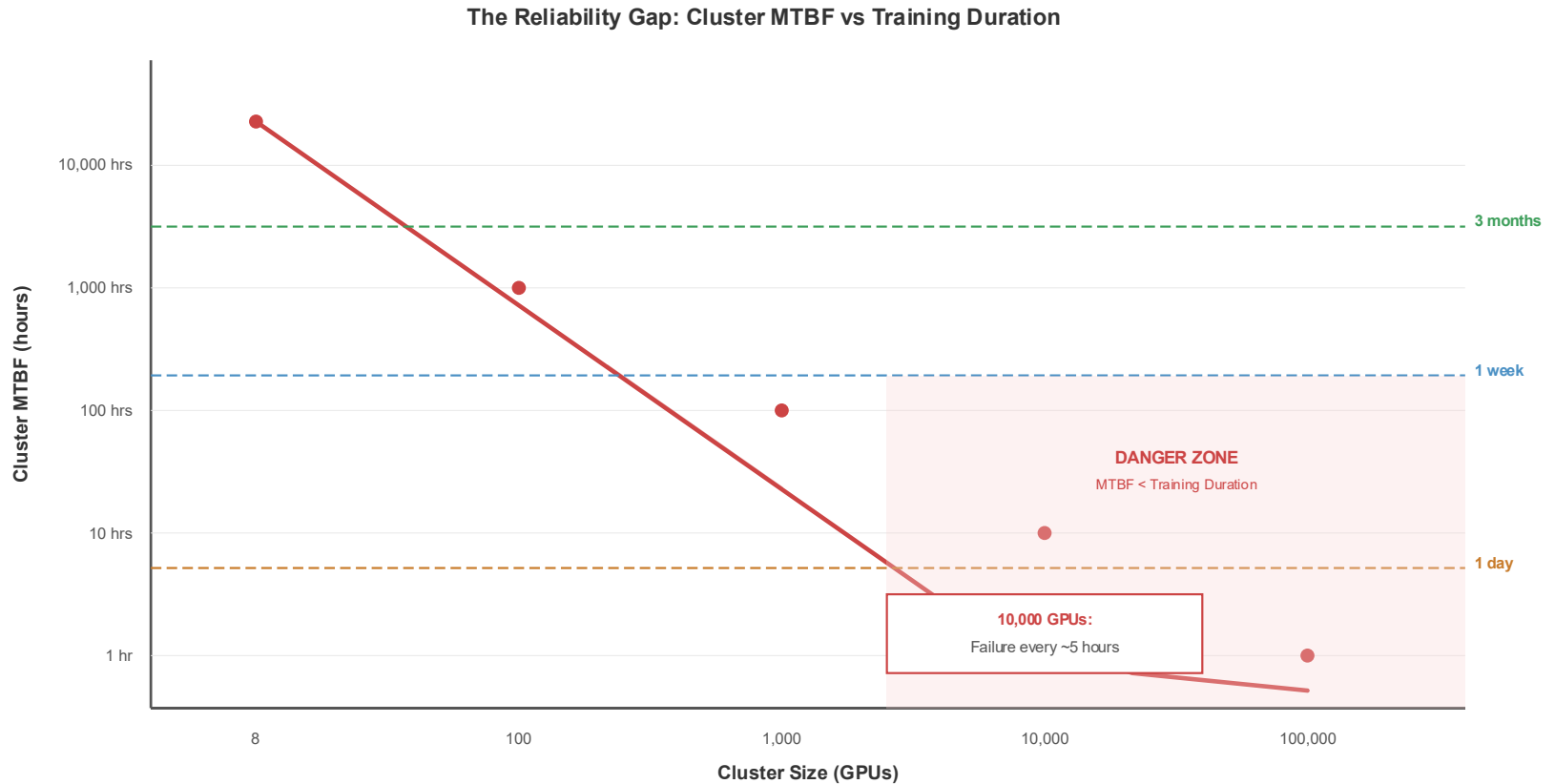
System MTBF:

$$MTBF_{system} = \frac{MTBF_{component}}{N}$$

Scale inverts reliability:

GPUs	Cluster MTBF
8	260 days
1,000	2 days
10,000	5 hours
25,000	2 hours
Per-GPU MTBF: 50,000 hours	

The reliability gap



Assumes per-GPU MTBF = 100,000 hours (server-grade). System MTBF = MTBF_component / N.

Implication: At 10K+ GPUs, fault tolerance is not optional – it is **physically necessary**.

Quick check

At 10,000 GPUs with 50,000h per-GPU MTBF, what is the cluster MTBF?

Predict: The 9s of reliability

Each GPU has **99.999%** uptime.

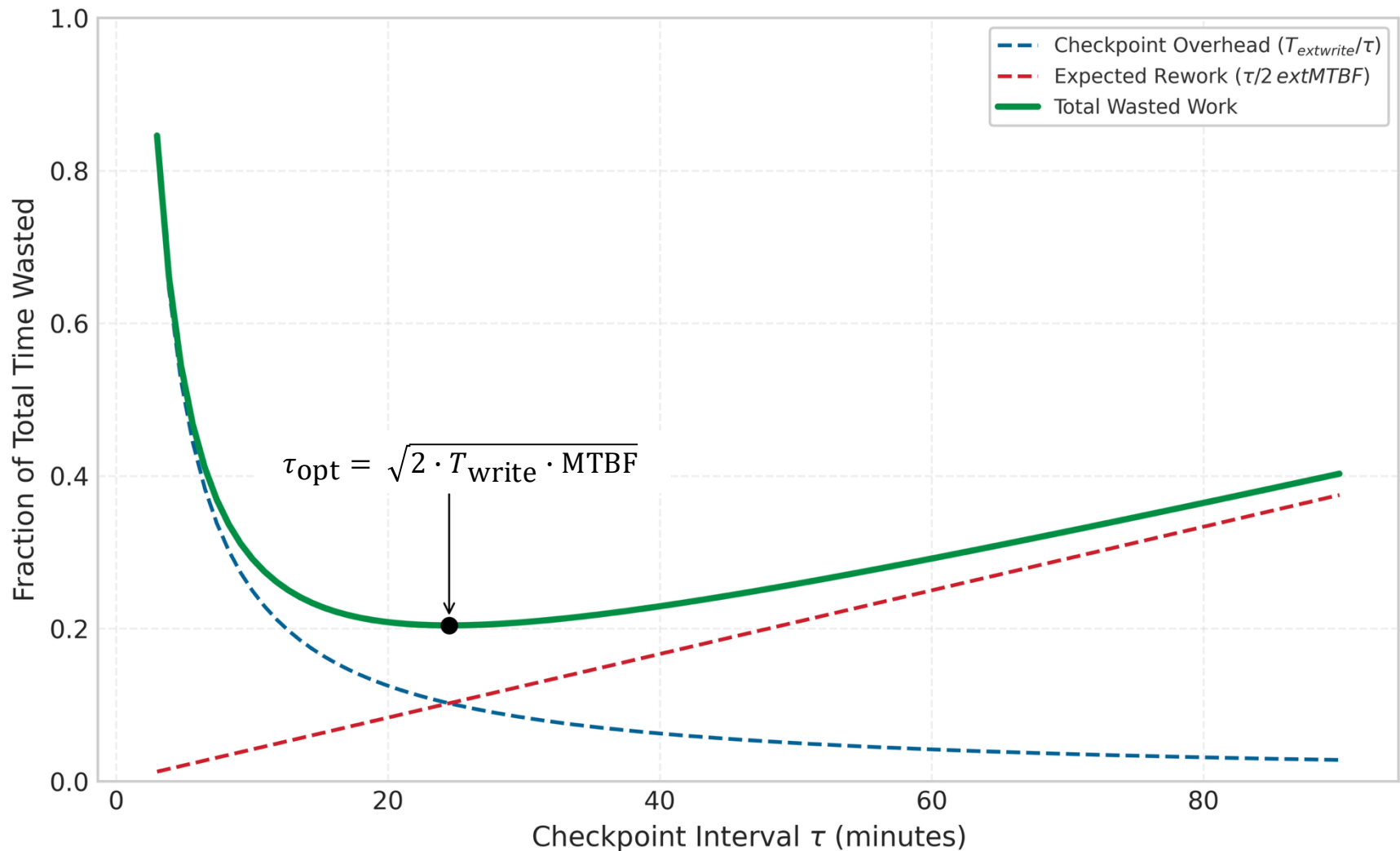
You have **10,000 GPUs**.

What is the probability the entire cluster survives **one hour** without a failure?

The Young-Daly Law

$$\tau_{\text{opt}} = \sqrt{2 \cdot T_{\text{write}} \cdot \text{MTBF}}$$

Optimal checkpoint interval



Young-Daly: Worked example

Scenario: Llama-3 training on 16,000 H100 GPUs

Parameter	Value	Source
Checkpoint write time (T_w)	2 min	Storage BW
Cluster MTBF	3 hrs (180 min)	16K GPUs

$$\tau_{\text{opt}} = \sqrt{2 \cdot 2 \cdot 180} = \sqrt{720} \approx \mathbf{27 \text{ min}}$$

Checkpoint every 10 min: 22.7% overhead (too frequent)

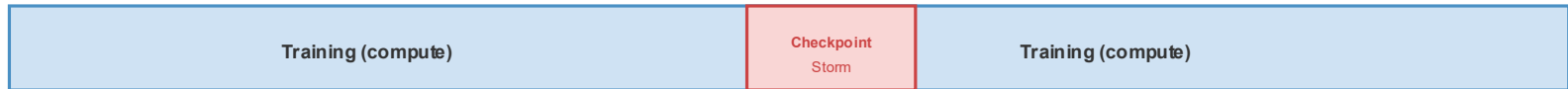
Checkpoint every 60 min: 20% overhead (too risky)

Checkpoint every 27 min: 14.9% overhead (optimal)

Checkpoint storage

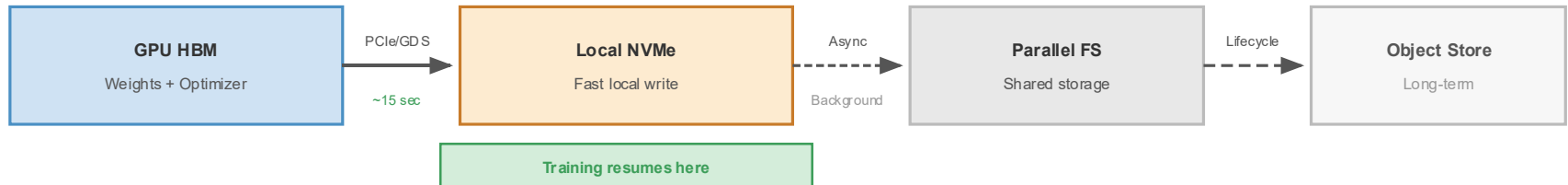
Checkpoint Staging: Minimizing T_{save}

Training Timeline with Checkpoint Storm



All 256 nodes write
1,050 GB simultaneously

Tiered Checkpoint Staging (Decouples T_{save} from PFS)



175B Model Checkpoint Math

Weights (FP16): 350 GB
Optimizer state (FP32): 1,400 GB
Total per checkpoint: 1,050 GB (with ZeRO)
30-day run: ~4.5 PB of checkpoint writes

Staging Impact

Without staging: $T_{\text{save}} = 150$ sec (PFS limited)
With NVMe staging: $T_{\text{save}} = 15$ sec
Training pause reduced by 10x
Async replication: zero impact on training

Checkpoint storm: All nodes write simultaneously. PFS must be provisioned for peak burst, not average. Write locally, replicate asynchronously.

The compute data path

Data lifecycle during a 30-day training run (checkpoint every 10 min):

Phase	Data Path	Volume
Setup	Object → NVMe (stage once)	3 TB
Training	NVMe → DRAM → HBM (per batch)	3 TB/epoch
Checkpoint	HBM → NVMe (local, fast)	1,050 GB each
Replicate	NVMe → PFS (async, background)	1,050 GB each
Archive	PFS → Object → Glacier (lifecycle)	~50 TB retained

Counterintuitive: Checkpoint I/O (~4.5 PB) dwarfs training data I/O (3 TB per epoch). For frontier models, **checkpoint staging strategy** has more impact on storage design than training data pipeline optimization.

Training failures in the wild

Training failures in the wild

Marin hero pretraining vs. Xiaomi MiMo-v2.6 agentic RL

Observed metric	Marin hero	MiMo Pro	MiMo Flash
Workload	Pretraining (535B A23B MoE), 18T tokens	Agentic RL	Agentic RL
Hardware	704× GB200	not disclosed	not disclosed
Wall time	38.1 d / 914 h	127.5 h	83.1 h
Step duration	15–20 s	2.8 h	2.2 h
Interruptions	37	14	5
MTBF (whole run)	24.7 h	9.1 h	16.6 h
Failure loss	55.9 h / 6.1%	30.1 h / 23.6%	13.7 h / 16.5%
Cost of loss	~39K GPU-h	~\$619K	~\$141K
Goodput	91.6%	~76%*	~84%*

The amount of work lost matters as much as failure frequency.

*MiMo goodput = 1 – loss fraction; excludes warm-up. Source: supplied analysis (Sep 26, 2026).

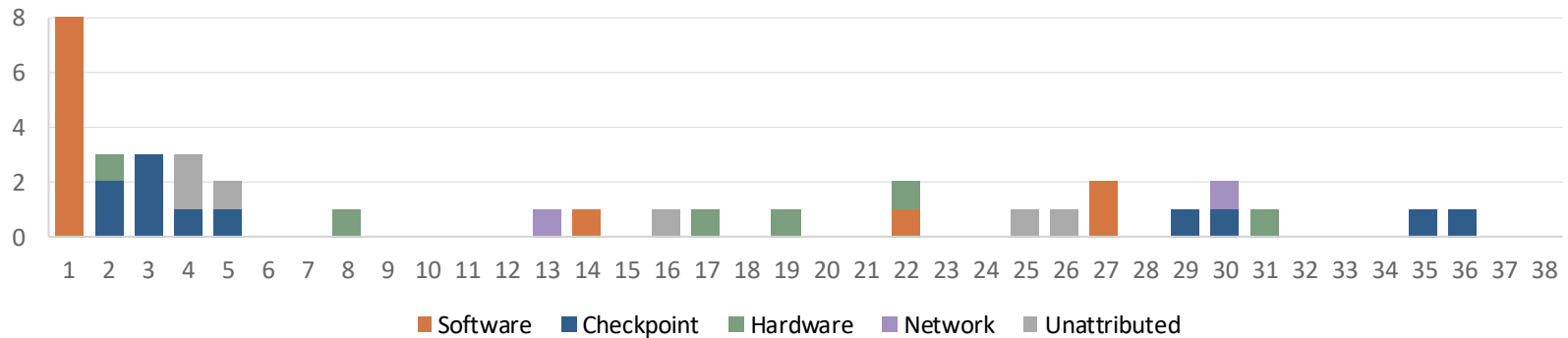
https://wandb.ai/marin-community/marin_moe/reports/535B-A23B-18T-Token-Hero-Run-Scaling-Ladder--VmlldzoxNzc2MDM5Ng

<https://mimo.xiaomi.com/rl/>

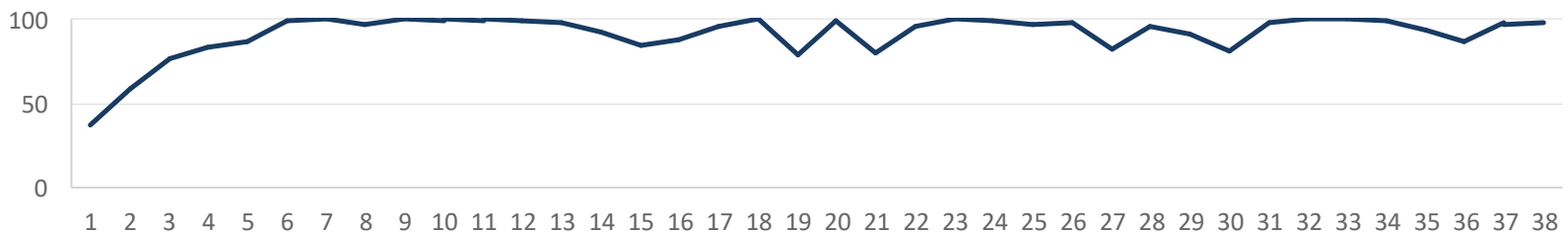
Marin: Early failures fade

Daily observations: Aug 20–Sep 26, 2026 (days 1–38).

Unplanned interruptions per day



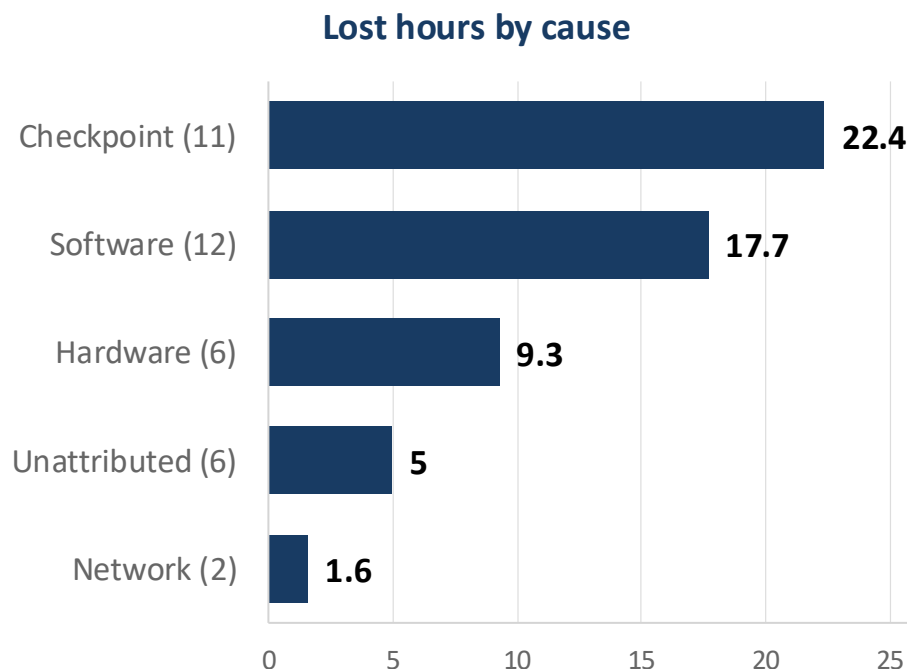
Daily goodput (%)



MTBF: 6.2 h in the first 4.9 days → 44 h in the next 33.2 days

Marin: Checkpoint system also fails

Checkpoint save, restore, and storage caused the most lost time.



One rank stalls the job

Aug 21: one restore hung
4 h 34 m. The other
703 GPUs waited.
Total loss: about 6 h.

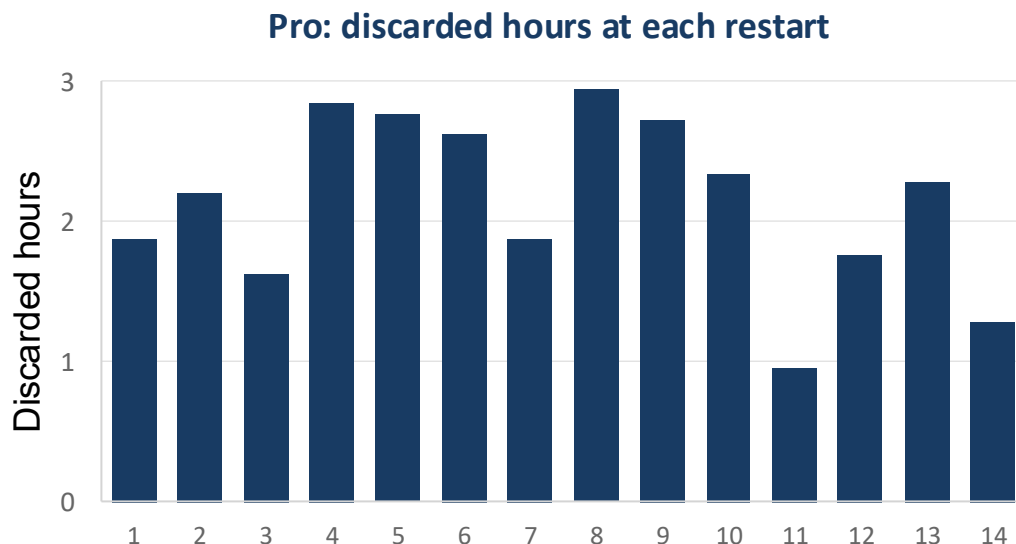
Retention exceeds quota

Sep 23: 104.56 TiB
against a 100 TiB quota.
Writes—and training—stop.

A checkpoint is useful only if it can be saved and restored.

MiMo RL: Expensive restarts

One restart discards a partial multi-hour RL step.



~\$760K lost

of ~\$3.47M combined spend

Pro: \$20.6K per hour
Flash: \$10.3K per hour

**20–40K sandboxes
per RL step**

Grader network outages restarted the trainer.
A silent Flash error forced rollback of 2 completed steps.

Preserve progress across rollouts, grading, and training so a local failure does not force the entire RL step to restart.

Young-Daly: Adapt to Marin's failure rate

Use the exposed checkpoint cost, not the background flush duration.

$$\tau^* \approx \sqrt{(2T_w M)} \quad W(\tau) \approx T_w/\tau + \tau/(2M) + R/M$$

M: MTBF T_w : exposed save cost R: restart cost

Scenario	M	T_w	τ^*	Used
Early life	6.2 h	7 min	~1.2 h	1 h
Steady phase*	44 h	2 min	~1.7 h	1 h

Early saves froze training for 6–8 min each hour.

Later saves flushed 4.29 TB in 96–120 s asynchronously.

Measure exposed overhead again after making saves asynchronous.

Young-Daly: What is an RL checkpoint?

Illustrative $T_w = 10$ min; the actual save cost is not disclosed.

Run	MTBF	Estimated τ^*	Step boundary
MiMo Pro	9.1 h	~1.7 h	~2.8 h
MiMo Flash	16.6 h	~2.4 h	~2.2 h

Pro: a shorter recovery interval may help.

Flash: step boundaries are already near this estimate.

Mid-step recovery must preserve more than model weights:

Policy + optimizer

Rollouts + rewards

Grader / sandbox state

Persist completed work, and isolate trainer–grader failures.

Marin: Budget retention, not just writes

Observed checkpoint: 4.29 TB; storage quota: 100 TiB $\approx$ 109.95 TB.

Capacity item	Illustrative retention policy
2 rolling checkpoints	8.58 TB
1 checkpoint being written	4.29 TB
17 permanent checkpoints	72.93 TB
Total retained / in flight	85.8 TB $\approx$ 78.0 TiB
Headroom within quota	$\sim$ 22%

Hourly saves write $\sim$ 103 TB/day; retention controls occupancy.

Keep every $\sim$ 28 h for 100 days? $\sim$ 369 TB of permanent copies.

Quota exhaustion is a retention-policy failure, not a bandwidth limit.